설비 고장을 예측하는 AI 데이터 분석

시계열 데이터 구조

시계열 데이터의 이해와 구조

posco

CONTENTS

목차

01 시계열 데이터란

02 두 가지 시계열 방식 — timestamp와 cycle

03 C-MAPSS 데이터 구조

04 실습 — C-MAPSS로 데이터 읽고 그래프 그리기

posco

01

시계열 데이터란

순서가 살아 있는 데이터의 정체와 설비 분석에서의 의미

순서가 있는 데이터와 없는 데이터

순서를 섞으면 정보가 사라지는 데이터가 시계열의 출발점

CASE 01 · 순서 무관

학생 키 목록

김철수 172, 이영희 165, 박민수 180

줄 순서를 바꿔도
우리 반 키 정보는 그대로 유지

CASE 02 · 순서 중요

1분마다 기록한 설비 온도

09:00 80.1 · 09:01 80.3 · 09:02 80.6

줄을 섞는 순간
"점점 올라가고 있었다" 정보가 소실

데이터에 순서가 의미를 가지는지 여부가 시계열 분석의 출발점

순서 무관 vs 순서 중요 데이터

관심사가 "값이 얼마"냐, "어떻게 변화"하느냐에 따라 분석 초점이 갈림

구분	순서 무관	순서 중요
예시	학생 키 목록	설비 온도
섞으면	의미 유지	정보 소실
핵심 질문	값이 얼마	어떻게 변화
분석 초점	평균 · 분포	추세 · 변동

설비 데이터는 오른쪽 — 추세와 변동을 보는 것이 분석의 본질

순서가 곧 시간 축

설비 데이터에서 순서의 정체는 거의 항상 시간

먼저 측정한 값이 앞, 나중 값이 뒤로 늘어선 시간 축

책 페이지나 레시피 단계도 순서가 중요하지만, 설비 데이터의 순서는 거의 항상 시간 축.

FIRST QUESTION

"이 데이터는 행의 순서가 의미를 가지는가"

그렇다면, 그리고 그 순서가 시간이라면, 시계열 분석으로 접근

데이터를 받았을 때 가장 먼저 던져야 할 질문

posco

시계열 데이터란 무엇인가

시간 순서대로 기록되고, 모든 값에 시간 정보가 함께 붙은 데이터

TIME SERIES DATA

시간이라는 축을 따라
한 줄로 늘어선 데이터

KEY 01 · 시간 축

"언제 측정됐는가" 시간 정보 부착

값만 있고 시간이 없으면 시계열이 아님

KEY 02 · 짝지음

모든 값에 시간 정보가 짝지어짐

하나의 시점에 하나의 값이 짝지음

posco

시간과 값이 한 쌍

시간 축과 측정값이 한 쌍씩 묶여 이어진 구조

TIME
09 : 00
09 : 01
09 : 02



VALUE (온도)
80.1
80.3
80.6

시간 정보 + 측정값 = 시계열

날씨 · 주가 · 미세먼지 · 심전도 — 모두 시간 축 위에 값이 늘어선 시계열

시계열 데이터의 종류

측정 간격과 변수 개수로 나눠 보면 시계열의 종류가 한눈에 정리

01 · 측정 간격

일정한가, 들쭉날쭉한가

1분마다 꼬박꼬박 vs
사건 있을 때만

설비 센서는 보통 일정, 통신 끊기면 간격 벌어짐

02 · 변수 개수

값 하나인가, 여러 개인가

단일 시계열 vs
다변량 시계열

온도 하나면 단일, 온도·압력·진동 함께면 다변량

03 · 설비 데이터

센서가 여러 개라 대부분 다변량

C-MAPSS는
센서 21개 다변량

실습 데이터는 다변량 시계열이라는 점을 미리 기억

측정 간격 · 변수 개수 두 축으로 시계열의 종류 정리

posco

설비 데이터가 시계열인 이유

설비가 쉬지 않고 돌며 시점·센서값이 시간 순서로 본질적으로 축적

모터 · 펌프 · 압축기 · 항공기 엔진

한번 켜지면 멈출 때까지 작동, 붙어 있는 온도·압력·진동 센서가
정해진 간격마다 값을 남기는 구조

INSIGHT

고장은 한순간에 일어나는 듯 보여도 데이터엔 흔적이 남음

그 흔적은 시간 순서로 봐야만 보이는 변화의 자취

posco

시계열이 만들어지는 흐름

설비 작동 → 센서 측정 → 시점 기록 → 시계열 축적

STEP 1

설비 작동

엔진·모터·펌프가 켜져 운전 시작

→

STEP 2

센서 측정

온도·압력·진동 등 물리량 측정

→

STEP 3

시점 기록

측정값에 "언제"를 짝지어 저장

→

STEP 4

시계열 축적

정해진 간격으로 반복, 시계열로 쌓임

한 시점만 보면 "정상 범위", 흐름으로 봐야 "올라가고 있음"의 위험 신호가 보임

posco

흐름으로 봐야 보이는 신호

한 시점의 값만 보면 정상 범위로 보이는 위험 신호

한 시점만 보기

"이 정도 진동은 정상인데?"

현재 값만 보면 위험 신호를 놓침

단일 시점의 한계

흐름으로 보기

같은 조건에서 흐름이 달라지는가

"정상 범위지만 계속 올라간다"는 위험 포착

시계열로 보는 진짜 이유

posco

시계열 분석으로 알 수 있는 것

추세 · 변동성 · 이상 징후 — 시계열 분석의 세 가지 목적지

01

추세

값이 전체적으로 어느 방향으로 가는가

02

변동성

값이 얼마나 흔들리는가

03

이상 징후

평소 패턴에서 벗어나는 순간

이후 학습 전체의 목적지를 한 줄로 — 세 가지 정보

세 가지 정보의 의미

추세 · 변동성 · 이상치 — 보통 이 순서로 신호가 강해짐

01 · 추세

값이 가는 방향

시간에 따라 올라가는지
내려가는지 전체 방향

엔진 온도가 점점 올라가면 부품 마모 신호일 가능성

가장 먼저 잡히는 약한 신호

02 · 변동성

값이 흔들리는 정도

값이 얼마나 출렁이는지
변동성 자체가 신호

고장 가까워지면 위아래로 크게 출렁이는 경우 많음

중간 단계의 신호

03 · 이상 징후

평소를 벗어난 튐

평소 패턴을 벗어난
갑작스러운 튐

범위를 한참 벗어난 값이 튀어 이상 의심 구간 탐지

가장 강한 신호 — 즉시 대응 필요

빨리 감지할수록 정비 시간 확보, 갑작스러운 고장 손실 감소

이후 학습의 목표 범위

데이터에서 추세 · 변동성 · 이상 징후를 직접 눈으로 찾는 능력이 목표

이후 학습에서 안 함

모델로 고장 예측까지 가지 않음

모델링은 이후 학습의 영역.
우선은 데이터에서 신호를 읽어내는 눈을 만듦.

이후 학습에서 함

세 가지 신호를 직접 찾는 능력

그래프로 추세 보기 · 이동표준편차로 변동성 재기
변화량으로 급변 지점 짚기

도구는 달라도 목적은 하나 — 정상에서 벗어나는 신호를 일찍 잡기

posco

정상 패턴과 이상 징후의 모습

잔잔한 정상에서 벗어나는 세 가지 모양 — 실제로는 섞여 나타남

정상

잔잔한 호수 수면

일정 수준 근처에서 잔잔하게 움직이는 흐름

추세 이상

한 방향으로 계속 밀려나는 형태

값이 시간 따라 우상향 혹은 우하향으로 이동

변동성 이상

흔들림이 갑자기 커지는 형태

값이 위아래로 크게 출렁이며 범위가 넓어짐

이상치

동떨어진 값이 튀는 스파이크

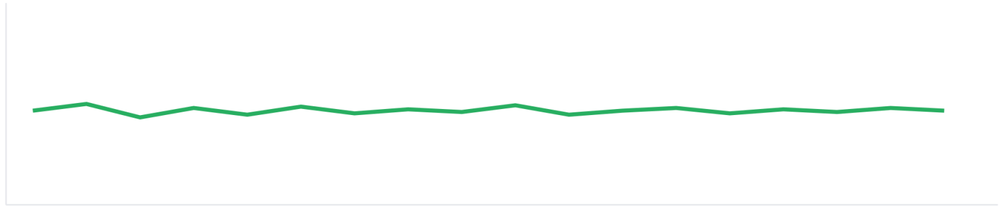
평소와 크게 다른 값이 한 점만 갑자기 튀어 오름

실제 설비에선 세 가지가 섞여 나타남

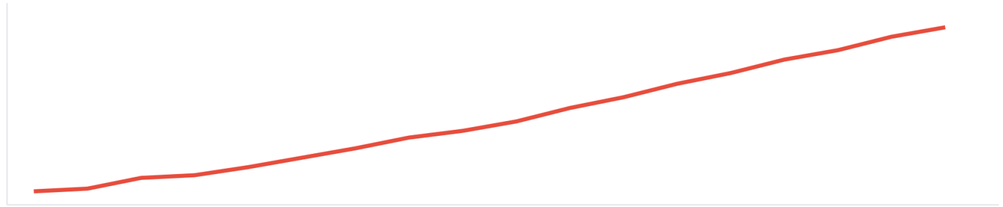
posco

네 가지 패턴 한눈에 보기

정상과 세 가지 이상 패턴을 그래프 모양으로 대비



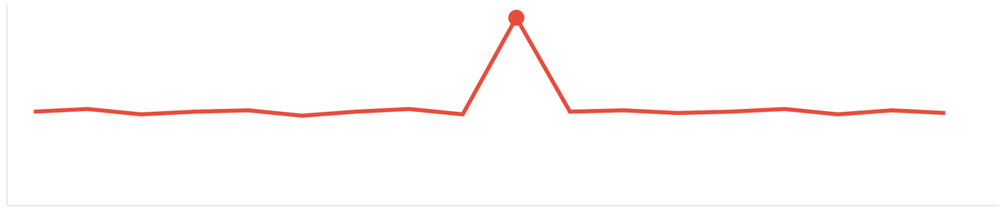
정상 · 잔잔한 가로 흐름



추세 이상 · 우상향 상승



변동성 이상 · 출렁임 점증



이상치 · 한 점만 위로 뾰족

네 가지 모양을 머릿속 그림으로 기억 — 분석은 보이는 패턴에 이름 붙이기에서 시작

posco

변화가 곧 고장은 아니다

판단 기준은 "평소 같은 조건의 패턴과 다른가"

모든 변화가 곧바로 고장을 뜻하지는 않음.

설비를 세게 가동하거나 날씨가 더우면 값은 자연히 변함.

같은 작동 조건인데도 흐름이 달라졌다면 그게 진짜 신호.

분석의 출발점

정상 패턴을 데이터로 충분히 익혀야 비정상이 보임

실습도 정상 흐름을 충분히 본 다음 벗어나는 지점을 찾는 순서로 진행

posco

02

두 가지 시계열 방식

timestamp와 cycle — 시간을 표현하는 두 방식

posco

timestamp 기반 시계열

실제 날짜와 시각이 시간 축으로 붙은 데이터 — 시점 도장이 찍힘

"2024년 3월 1일 오전 9시 00분"처럼 시점 도장

01 · 장점

"언제"가 명확

시 · 일 · 월 단위 집계 가능
시간대별 패턴 분석에 강함

02 · 대표 데이터

공장 · 발전소 · 건물 에너지

현장 데이터 대부분이 이 방식
실시간 모니터링과 운영 분석에 적합

"하루 중 몇 시에 온도가 가장 높은가" 같은 질문에 답할 수 있는 방식

timestamp 데이터의 실제 모습

왼쪽 시간 열이 실제 날짜·시각인 점이 핵심

timestamp	온도
2024-03-01 09:00:00	80.1
2024-03-01 09:01:00	80.3
2024-03-01 09:02:00	80.6

건강 앱 심박수도 같은 방식 — "오후 3시 12분 심박수 78"

CSV에서 막 읽은 날짜는 그냥 글자 — 시간으로 인식시키는 변환 필요

"운동 시간대에 높음", "잘 때 가장 낮음" 같은 분석이 가능한 방식

posco

cycle 기반 시계열

실제 시각 대신 "몇 번째 작동인가" 번호가 시간 축

DEFINITION

한 번의 작동 주기 — 비행기가 한 번 이륙해 착륙할 때까지가 한 cycle

KEY 01

실제 날짜·시각이 없음

시 · 일 · 월 집계는 그대로 적용할 수 없음

KEY 02

NASA C-MAPSS가 바로 이 방식

1번째 비행 · 2번째 비행 — 작동 횟수로 순서를 매김

작동 횟수를 시간처럼 다루는 시계열

posco

왜 cycle을 쓰는가

자동차 주행거리처럼 "몇 km 탔나"로 판단 — cycle도 같은 발상

01 · 마모 비례

달력 시간이 아닌 작동 횟수

닳고 망가짐은
작동 횟수에 비례

한 달에 한 번 vs 하루 백 번 — 같은 한 달이라도 닳음 정도가 다름

자동차 주행거리와 같은 발상

02 · 공정 비교

여러 설비를 같은 척도로

작동 횟수 기준으로
설비 간 비교

가동 빈도가 달라도 같은 척도로 닳음 정도 비교 가능

공정 단위 비교에 유리

03 · 순서가 시간

1, 2, 3 번호도 시간 흐름

분명한 순서가 곧
시간 흐름

단위가 초·분·시가 아니라 작동 횟수일 뿐

핵심은 순서가 살아 있는가

cycle 데이터의 실제 모습

왼쪽 열이 날짜가 아니라 단순 작동 번호 — time_cycles를 가로축에 둠

time_cycles	온도
1	1399.2
2	1399.3
3	1395.0

이동평균은 "최근 몇 번 작동" 단위, 변화량은 "직전 작동 대비 차이"

시간 단위만 다를 뿐 본질은 timestamp와 동일

왼쪽이 단순 번호 — 이게 cycle 데이터의 정체

posco

timestamp와 cycle 비교

공통점과 차이점 — 두 방식의 본질과 차이를 한눈에

Q1. **두 방식의 공통점은**

둘 다 순서가 살아 있는 시계열, 섞으면 정보 소실

추세 · 변동 · 이상 탐지 목적도 동일

Q2. **두 방식의 차이점은**

시간 축이 실제 시각이냐, 작동 횟수냐

그에 따라 쓸 수 있는 도구가 달라짐

cycle 데이터로는 시·일·월 집계 연습 불가 — 강사 제공 timestamp 변환 샘플로 보충

posco

두 방식 핵심 차이 정리

시간 축의 차이가 쓸 수 있는 도구의 차이로 이어짐

구분	timestamp	cycle
시간 축	날짜 · 시각	작동 번호
예시	2024-03-01 09:00	137번째
분석	시 · 일 집계	이동평균
데이터	공장 센서	C-MAPSS

"1 cycle = 1시간"으로 약속하면 cycle 데이터를 timestamp로 변환 가능

posco

학습 순서 안내

두 시간 축을 오가며 어떤 시계열에도 알맞은 도구를 고르는 힘

STEP 1

C-MAPSS로 시작

cycle 데이터로 데이터 구조와 변화 익히기

이번 시간

→

STEP 2

timestamp 샘플

시간 인덱스와 시·일·월 집계 배우기

다음 시간

→

STEP 3

다시 cycle로

이동평균 · 변화량 · 상관 분석

완성 단계

cycle → timestamp → cycle 흐름으로 두 축을 오가며 학습

posco

설비 PdM에서 두 방식의 활용

어느 방식이 더 좋은가가 아니라 어떤 질문에 답하려는가가 선택 기준

01 · 모니터링

timestamp 활용 — 지금 상태가 정상인지 실시간 확인

실시간 감시와 운영 분석에 강함 — 관제 화면, 일일 보고서

"지금 상태가 어떤가" · "어젯밤 온도가 비정상이었나"

02 · 수명 예측

cycle 활용 — 앞으로 몇 번 더 쓸 수 있는지

남은 수명 예측에 강함 — 이후 학습의 RUL 개념도 cycle 기반

"200번 돌았는데 고장까지 얼마 남았나"

좋고 나쁨이 아닌 성향 차이 — 답할 질문에 따라 선택

posco

현장 활용 목적 구분

감시·운영은 timestamp, 수명·비교는 cycle이 자연스러움

목적	방식	대표 질문
실시간 감시	timestamp	지금 정상인가
운영 분석	timestamp	시간대 차이
수명 예측	cycle	몇 번 더 사용
설비 비교	cycle	어느 게 더 닳음

예지보전은 timestamp로 감시 · cycle로 수명 가늠 — 가장 똑똑한 정비 방식

posco

정비 방식 세 가지

가장 똑똑한 예지보전을 가능하게 하는 것이 시계열 데이터 분석

STEP 1

사후 정비

고장 난 다음에 고침

갑작스러운 중단 발생

→

STEP 2

예방 정비

일정대로 미리 점검

멀쩡한 부품도 교체해 낭비

→

STEP 3

예지보전

데이터로 필요한 시점에 정비

낭비와 고장 모두 감소

우리가 배우는 모든 것은 결국 예지보전을 위한 기초 체력

posco

03

C-MAPSS 데이터 구조

pandas로 NASA 항공기 엔진 데이터 열고 살펴보기

posco

시계열을 다루는 도구 pandas

표 형태 데이터를 코드로 다루는 파이썬 도구 — 시계열 기능이 특히 강력

METAPHOR

엑셀은 마우스로, pandas는 코드로 같은 표를 다루는 일

그릇

데이터프레임 (DataFrame)

표 하나를 담는 그릇 — CSV가 데이터프레임으로 메모리에 올라옴

흐름

열기 → 살펴보기 → 계산 → 그래프

모든 실습이 이 흐름으로 반복 — 몸에 배도록

코드로 다루면 같은 작업 수백 번 반복도, 수십만 줄 처리도 쉬워짐

posco

시계열에서 쓸 주요 도구 — 데이터 준비

pandas는 잘 갖춰진 부엌 — 칼·냄비가 다 준비되어 있음

01 · READ_CSV

파일을 메모리로

CSV를
데이터프레임으로 적재

모든 실습의 첫 줄 — 데이터를 코드 세계로 끌어옴

pd.read_csv('파일명')

02 · TO_DATETIME

글자를 시간으로

글자 날짜를
시간으로 인식

"2024-03-01"이라는 문자를 시간 타입으로 변환

pd.to_datetime(...)

03 · SET_INDEX

시간을 기준 축으로

시간을
기준 인덱스로

시간 단위 집계의 발판 — 시계열 분석의 출발선

df.set_index(...)

파일 열기 → 시간 인식 → 시간 인덱스 — 데이터 준비 3종 세트

posco

시계열에서 쓸 주요 도구 — 분석

데이터 준비가 끝나면 분석 도구로 추세를 부각

04 · RESAMPLE

시간 단위 집계

시 · 일 · 월 단위로 묶어 평균 집계

1분 단위 측정값을 시간별·일별로 압축해 흐름을 명확히 봄

timestamp 방식의 대표 도구

05 · ROLLING

이동평균으로 추세 부각

이동평균으로 흐름을 매끄럽게

잡음을 평탄화해 진짜 추세만 도드라지게 만듦

cycle · timestamp 둘 다에서 쓸 수 있는 만능 도구

NumPy는 숫자 배열, pandas는 표 구조, matplotlib·seaborn은 그래프

posco

CSV를 코드로 여는 흐름

pd.read_csv 한 줄로 데이터프레임이 되어 메모리에 올라옴

import pandas as pd로 pd라는 짧은 이름으로 호출

import는 보통 파일 맨 위에 한 번만 적으면 됨 — 매번 적을 필요는 없음

자주 하는 실수

파일 이름 오타 · 다른 폴더에 둔 경우

파일과 코드가 같은 폴더에 있는지 먼저 확인하는 습관

posco

CSV 여는 기본 코드

파일을 연 직후엔 head로 잘 열렸는지 꼭 확인

CODE

```
import pandas as pd

df = pd.read_csv('cmapss_fd001_sample.csv')

# 첫 5줄로 잘 열렸는지 즉시 확인
print(df.head())
```

"열고 나면 확인한다" — 이 한 쌍을 항상 함께

posco

데이터 구조 확인 메서드

데이터프레임을 처음 열면 가장 먼저 확인하는 네 가지

1	`head`	맨 앞 5줄 모습 확인 — 열 구성과 값의 모양을 한눈에
2	`info`	행·열·자료형·결측 요약 — 데이터의 건강검진표
3	`describe`	평균·최대·최소 통계 — 값의 범위와 분포 파악
4	`shape`	행 개수, 열 개수 확인 — 데이터의 규모 점검

head는 괄호가 붙고 shape · columns는 괄호 없음 — 차이 기억

네 가지 도구 정리

shape → head → info → describe 순서로 데이터를 둘러보기

도구	코드	알 수 있는 것
head	`df.head()`	맨 앞 5줄
info	`df.info()`	자료형 · 결측
describe	`df.describe()`	평균 · 최대
shape	`df.shape`	행 · 열 개수

posco

C-MAPSS FD001 데이터 소개

NASA가 시뮬레이션으로 만든 예지보전 표준 데이터셋

NASA C-MAPSS FD001

"NASA가 만든 엔진 고장 데이터"
예지보전 공부의 표준

핵심 특징

엔진이 고장 날 때까지의 전 과정이 통째로 담김

정상에서 고장까지 가는 변화 — 추세와 변동성 공부에 최적

항공기 엔진 시뮬레이션 데이터 — 잡음과 개체차도 포함된 현실적 모사

정상에서 고장까지의 흐름

cycle이 늘수록 점진적으로 열화하는 엔진 수명 흐름

초기 cycle



정상 작동



cycle 증가에 따른 점진적 열화

센서값이 서서히 변하는 곡선

마지막 cycle



고장

SIMULATION

실제 엔진을 고장까지 돌리는 비용·위험 → 물리 법칙 시뮬레이터로 생성

측정 잡음과 개체차도 포함 — FD001~FD004 중 가장 단순한 FD001 교육 샘플 사용

입문용 크기로 다듬은 교육 샘플 — 진짜 설비 연습에 적합

posco

FD001 컬럼 구조 이해

총 26개 열, 네 묶음으로 나누면 이해 쉬움

1	**엔진 번호** `unit_nr`	어느 엔진의 기록인가 — 여러 엔진 구분
2	**작동 cycle** `time_cycles`	몇 번째 작동인가 — cycle 기반 시간 축
3	**운전 설정** `setting_1~3`	작동 조건 — 같은 조건의 패턴인지 판단할 때 활용
4	**센서값** `s_1~s_21`	온도 · 압력 · 진동 등 21개 — 분석의 핵심

21개 중 거의 안 변하는 것도, 뚜렷이 변하는 것도 있음 — 변하는 센서를 골라내는 것이 분석

posco

컬럼 네 묶음 정리

s_4 · s_11은 cycle 증가에 따라 상승, s_7 · s_20은 하락하도록 구성한 샘플

묶음	열 이름	의미
엔진 번호	`unit_nr`	어느 엔진
작동 cycle	`time_cycles`	몇 번째
운전 설정	`setting_1~3`	작동 조건
센서값	`s_1~s_21`	온도 · 압력

의미 없이 번호만 — 패턴만으로 좋은 센서를 판단하는 훈련이 됨

엔진 1대를 cycle 순으로 보는 관점

한 엔진을 고르고 cycle 순으로 정렬하는 두 단계가 핵심

시계열 분석의 기본 — 엔진 한 대를 골라서 보기

1번 엔진만 떼어내면 첫 작동부터 마지막까지 깔끔하게 따라갈 수 있는 수명 곡선

주의

여러 엔진을 한 그래프에 섞으면 흐름이 깨짐 — 반드시 한 엔진씩

습관

"보기 전에 정렬한다"를 습관으로 — sort_values로 cycle 순 정리

한 엔진 보는 단계

전체데이터 → 엔진필터 → cycle정렬 → 변화관찰

STEP 1

전체 데이터

여러 엔진이 섞인 원본 데이터프레임

→

STEP 2

엔진 필터

df[df['unit_nr']==1]로 1번 엔진 선택

→

STEP 3

cycle 정렬

sort_values로 cycle 순서 정렬

→

STEP 4

변화 관찰

cycle에 따른 센서값 변화 추적

"대상을 고르고 시간 순으로 정렬" — 거의 항상 함께 붙어 다니는 두 단계

posco

시계열 그래프 읽는 법

방향 · 떨림 · 튐 — 추세 · 변동성 · 이상치와 정확히 대응

01 · 그래프 구조

가로축은 시간, 세로축은 측정값

cycle 또는 시각이 가로축에 배치

02 · 방향 보기

우상향 → 상승, 우하향 → 하락

추세를 보는 첫 단계

03 · 떨림 보기

잔잔하면 안정, 크게 출렁이면 변동성 큼

변동성을 눈으로 가늠하는 단계

04 · 튐 보기

혼자 동떨어진 점은 이상치 의심

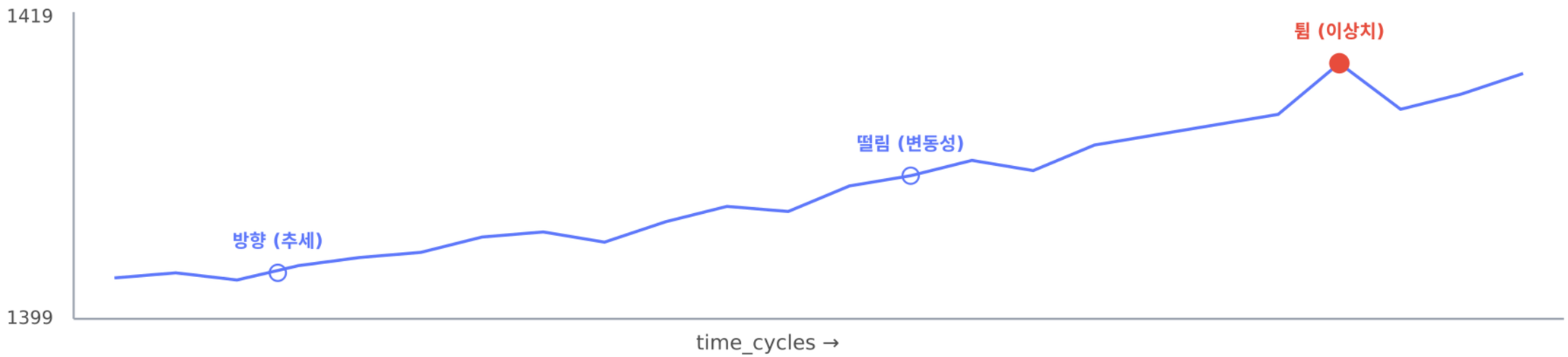
이상 징후를 짚어내는 단계

그렸으면 방향 · 떨림 · 튐 순서로 확인

posco

그래프 읽기 포인트

그래프 인상만 믿지 말고 세로축 숫자를 함께 읽기



세로축 범위에 속지 않기 — 1399~1419에서 화면 가득 출렁이면 실제 변화 폭은 단 20

제목과 축 이름은 반드시 — 미세 변화는 범위 좁히기, 절대 크기는 범위 넓히기

같은 그래프라도 세로축에 따라 다르게 보임

posco

센서값의 단위와 범위 이해

센서마다 측정 물리량이 달라 값의 크기와 범위가 제각각

값 **9000 센서**와 **8 센서**를 같은 축에 그리면 작은 값이 안 보임.

변화 판단도 원래 값 대비 얼마나 변했는지 함께 봐야 함.

문제

8짜리는 바닥에 납작 깔림

단위 다른 센서는 따로 그리거나 정규화 필요

해법 기본기

describe로 범위 먼저 확인

변화율은 pct_change로 따로 살펴봄

9000짜리 10 변화 ≠ 8짜리 10 변화 — 의미가 다름

posco

센서별 값 범위 예시

s_9는 9000 이상, s_15는 8 근처 — 한 그래프에 겹치면 s_15가 안 보임

센서	대략 범위	성격
s_2	**640 근처**	온도 계열
s_4	**1400 근처**	온도 계열
s_9	**9000 이상**	회전수
s_15	**8 근처**	비율 계열

평균 빼고 표준편차로 나눠 같은 척도로 맞추는 정규화·표준화는 이후 학습에서

결측·이상값이 시계열에 미치는 영향

시계열에서는 흐름이 깨지기 때문에 더 골치 아픈 두 가지 문제

01 · 결측치

값이 비어 있는 칸

센서 멈춤 · 통신 끊김으로 발생

흐름이 끊김 — 이동평균에 빈 값이 끼면 어긋남

isna로 빈 칸 개수부터 세기

02 · 이상값

평소와 동떨어진 값

센서 오작동으로 발생

주변 평균 · 변동 흔듦 — 통계가 통째로 흔들림

describe로 비정상 값 점검

흐름이 끊긴 자리를 모르고 분석하면 결과를 믿을 수 없음

posco

결측 확인 기본 코드

분석 전에 결측과 이상부터 확인하는 습관

CODE

```
# 각 열의 결측치 개수 확인
print(df.isna().sum())

# 범위와 분포로 비정상 값 점검
print(df.describe())
```

시계열 결측은 앞뒤 값으로 자연스럽게 잇는 보간(interpolation)으로 처리

전체 평균으로 채우면 흐름이 부자연스럽게 끊김 — 단원 뒤쪽에서 자세히

isna로 결측 세고, describe로 비정상 값 점검

posco

분석에 쓸 센서 고르기

거의 안 변하는 센서는 빼고, 뚜렷이 변하는 센서를 선택

s_1은 항상 518.67, **s_5**는 항상 14.62 — 정보가 없어 제외

s_4는 1399 → 1419 상승 — 추세 분명해 핵심 대상

선별법 01

describe로 표준편차 보기

0에 가까우면 제외, 크면 활용

선별법 02

첫 cycle과 마지막 cycle 비교

값 차이가 크면 열화 반영 가능성

항상 똑같은 센서는 과감히 빼기

센서 구분 예시

"값이 변하는가"와 "방향이 일관적인가"를 함께 봐야 함

센서	변화 여부	활용
s_1	항상 518.67	제외
s_5	항상 14.62	제외
s_4	상승 추세	활용
s_11	점진 상승	활용

표준편차 작아도 한 방향으로 꾸준히 밀려나는 센서는 좋은 신호일 수 있음

posco

데이터를 보는 3가지 질문

크기 → 구조 → 상태 순으로 좁혀 가는 데이터 분석가의 첫 행동 습관

STEP 1

얼마나 큰가

shape와 columns로 행·열 개수와 이름 확인

`df.shape · df.columns`

STEP 2

어떻게 생겼나

head와 info로 값 모습과 자료형·결측 확인

`df.head() · df.info()`

STEP 3

값들이 정상인가

describe와 isna로 범위·평균·빈 값 점검

`df.describe() · df.isna()`

어떤 데이터를 받아도 이 세 질문부터 — 거꾸로 값부터 보면 길을 잃기 쉬움

세 질문 점검 체계

설비 데이터뿐 아니라 어떤 데이터에도 똑같이 적용

01 · 크기

shape · columns

데이터가 몇 줄인지
전체 규모 파악

1116행인지 100만 행인지에 따라 마음가짐이 다름

가장 먼저 던지는 질문

02 · 구조

head · info

어떤 열이 숫자·글자인지
자료형 확인

숫자여야 할 열이 글자로 읽히지 않았는지 점검

분석 가능한 형태인지 확인

03 · 상태

describe · isna

범위·평균과
결측 여부 점검

분석 가능한 상태인지 마지막 점검

한두 줄 메모로 남기기

"1116행 26열, 엔진 5대, 결측 없음, s_4와 s_11 상승 추세" — 분석을 잘하는 사람의 공통점

04

실습 — C-MAPSS로 직접 손에 익히기

데이터 열기 · 통계 · 그래프 · 미니 리포트까지 단계별 실습

실습 1. C-MAPSS 데이터 열고 구조 확인

샘플을 직접 열어 크기와 열 구성을 파악

목표

강사가 준 엔진 데이터를 직접 열어 크기와 열 구성을 파악

단계

· pandas를 불러와 샘플 파일을 데이터프레임으로 열기

· 크기 속성으로 행·열 개수를, 열 이름 속성으로 컬럼을 확인

· 앞부분을 미리 보며 값의 모양을 눈으로 익히기

예상 결과

행 천여 개·스물여섯 열, 엔진번호·작동회차·센서 열이 보임

posco

실습 1. C-MAPSS 데이터 열고 구조 확인 정답

샘플을 직접 열어 크기와 열 구성을 파악

CODE · 실습 1

```
import pandas as pd
df = pd.read_csv('cmapss_fd001_sample.csv')
print(df.shape)                         # (1116, 26)
print(df.columns.tolist()[:5])
print(df[['unit_nr', 'time_cycles', 's_2', 's_11']].head())
```

실습 2. 엔진 단위와 작동 길이 파악

여러 엔진이 섞인 구조와 엔진별 작동 길이를 확인

목표

데이터에 여러 엔진이 섞여 있음을 확인하고 엔진별 작동 길이를 파악

단계

· 엔진번호의 고유값 개수로 엔진이 몇 대인지 세기

· 엔진번호별 행 수를 세어 각 엔진의 작동 회차 길이를 구하기

· 번호 순으로 정렬해 엔진마다 길이가 다름을 확인

예상 결과

엔진 다섯 대, 엔진별 회차 수가 179~287로 서로 다름

posco

실습 2. 엔진 단위와 작동 길이 파악 정답

여러 엔진이 섞인 구조와 엔진별 작동 길이를 확인

CODE · 실습 2

```
print('엔진 대수:', df['unit_nr'].nunique())      # 5
cnt = df['unit_nr'].value_counts().sort_index()
print(cnt)
# 1:192  2:287  3:179  4:189  5:269  (엔진마다 작동 길이가 다름)
```

posco

실습 3. 요약 통계로 센서 범위 읽기

평균·표준편차·최소·최대로 센서 범위를 수치로 파악

목표

요약 통계로 주요 센서의 평균과 값 범위를 수치로 파악

단계

· 2번 엔진을 골라 세 센서의 요약 통계를 한 번에 보기

· 평균으로 대략 크기를, 흔들림 폭으로 변동 정도를 읽기

· 최소·최대로 각 센서의 값 범위가 서로 다름을 확인

예상 결과

세 센서의 평균이 약 641.8·47.7·517.4로 크기가 제각각

posco

실습 3. 요약 통계로 센서 범위 읽기 정답

평균·표준편차·최소·최대로 센서 범위를 수치로 파악

CODE · 실습 3

```
e2 = df[df['unit_nr'] == 2]
print(e2[['s_2', 's_11', 's_12']].describe().round(2))
# mean   641.75 / 47.69 / 517.43   (센서마다 크기·범위가 다름)
```

posco

실습 4. 변하는 센서 선별과 결측 확인

거의 안 변하는 센서를 빼고 쓸 센서를 직접 고르기

목표

거의 변하지 않는 센서를 빼고 분석에 쓸 센서를 직접 선별

단계

· 여러 센서의 흔들림 폭을 비교해 0에 가까운 것은 제외 대상으로 표시

· 뚜렷이 변하는 센서를 분석 대상으로 남기기

· 결측 합계를 세어 빈 값이 없는지 확인

예상 결과

첫 두 센서는 값이 늘 같아 제외, 뒤 센서는 변함, 결측 없음

posco

실습 4. 변하는 센서 선별과 결측 확인 정답

거의 안 변하는 센서를 빼고 쓸 센서를 직접 고르기

CODE · 실습 4

```
for c in ['s_1', 's_6', 's_7', 's_11', 's_12']:
    print(c, round(df[c].std(), 3))
# s_1 0.0  s_6 0.0  s_7 2.367  s_11 0.314  s_12 2.119
print('결측 합계:', int(df.isna().sum().sum()))     # 0
```

실습 5. 엔진 하나 골라 첫·끝 추세 읽기

한 엔진을 회차 순으로 정렬해 첫·끝 값으로 방향 판정

목표

엔진 하나를 골라 회차 순으로 정렬하고 첫·끝 값으로 추세 방향을 읽기

단계

· 2번 엔진만 골라 작동 회차 순으로 정렬

· 상승 센서와 하강 센서의 첫 회차 값과 마지막 값을 꺼내기

· 첫 값과 끝 값을 비교해 상승인지 하강인지 판정

예상 결과

한 센서는 47.30에서 48.18로 상승, 다른 센서는 하강

posco

실습 5. 엔진 하나 골라 첫·끝 추세 읽기 정답

한 엔진을 회차 순으로 정렬해 첫·끝 값으로 방향 판정

CODE · 실습 5

```
e2 = df[df['unit_nr'] == 2].sort_values('time_cycles')
for c in ['s_11', 's_7']:
    a, b = e2[c].iloc[0], e2[c].iloc[-1]
    print(c, a, '->', b, '상승' if b > a else '하강')
# s_11 47.3 -> 48.18 상승,   s_7 553.84 -> 546.17 하강
```

posco

실습 6. 엔진 센서 시계열 그래프 그리기

정렬한 엔진 데이터로 회차 가로축 선 그래프 그리기

목표

정렬한 엔진 데이터로 회차를 가로축, 센서를 세로축으로 선 그래프 그리기

단계

· 상승 센서를 선 그래프로 그리고 제목·축 이름 붙이기

· 하강 센서도 같은 방식으로 그려 방향을 비교

· 두 그래프의 기울기 방향으로 추세를 눈으로 확인

예상 결과

한 센서는 우상향, 다른 센서는 우하향 곡선

posco

실습 6. 엔진 센서 시계열 그래프 그리기 정답

정렬한 엔진 데이터로 회차 가로축 선 그래프 그리기

CODE · 실습 6

```
import matplotlib.pyplot as plt
plt.plot(e2['time_cycles'], e2['s_11'])
plt.title('Engine 2 - s_11'); plt.xlabel('time_cycles')
plt.show()                                # 우상향(상승 추세)
plt.plot(e2['time_cycles'], e2['s_12'])
plt.title('Engine 2 - s_12'); plt.show()   # 우하향(하강 추세)
```

posco

실습 7. 여러 센서 비교와 범위 문제

여러 센서를 함께 그려 값 범위 차이 문제를 체험

목표

여러 센서를 함께 그려 비교하고 값 범위가 다를 때의 문제를 체험

단계

· 크기가 비슷한 두 센서를 함께 그리고 범례를 붙여 비교

· 크기가 크게 다른 두 센서를 일부러 겹쳐 그리기

· 두 센서의 범위를 숫자로 확인해 작은 값이 왜 안 보이는지 이해

예상 결과

약 8100 센서에 가려 약 38 센서가 평평하게 보임

posco

실습 7. 여러 센서 비교와 범위 문제 정답

여러 센서를 함께 그려 값 범위 차이 문제를 체험

CODE · 실습 7

```
plt.plot(e2['time_cycles'], e2['s_2'], label='s_2')
plt.plot(e2['time_cycles'], e2['s_7'], label='s_7')
plt.legend(); plt.show()                # 비슷한 범위: 둘 다 보임
plt.plot(e2['time_cycles'], e2['s_14'], label='s_14')
plt.plot(e2['time_cycles'], e2['s_20'], label='s_20')
plt.legend(); plt.show()                # 범위 차이 큼
print(round(e2['s_14'].min(), 1), round(e2['s_14'].max(), 1))  # 8099.1 8176.4
print(round(e2['s_20'].min(), 2), round(e2['s_20'].max(), 2))  # 37.88 39.06
```

posco

실습 8. 데이터 구조 미니 리포트

오늘 확인한 크기·엔진·센서 특성을 짧은 리포트로 정리

목표

오늘 확인한 크기·엔진·센서 특성을 짧은 리포트로 자동 정리

단계

· 전체 크기와 엔진 대수를 한 줄로 출력

· 대상 엔진의 작동 회차 수와 결측 유무를 정리

· 주목 센서 세 개의 첫·끝 방향을 반복문으로 표처럼 정리

예상 결과

크기·엔진 수·회차·결측·센서 방향이 담긴 짧은 명세 리포트

posco

실습 8. 데이터 구조 미니 리포트 정답

오늘 확인한 크기·엔진·센서 특성을 짧은 리포트로 정리

CODE · 실습 8

```
target = 2
g = df[df['unit_nr'] == target].sort_values('time_cycles')
print(f'크기: {df.shape[0]}행 {df.shape[1]}열, 엔진 {df["unit_nr"].nunique()}대')
miss = '없음' if df.isna().sum().sum() == 0 else '있음'
print(f'엔진 {target} 회차 수: {len(g)}, 결측: {miss}')
for c in ['s_11', 's_7', 's_12']:
    a, b = g[c].iloc[0], g[c].iloc[-1]
    print(f'  {c}: {a} -> {b} ({"상승" if b > a else "하강"})')
```